



Taller grupal

analítica de datos

Juan Felipe Rodríguez G. 20261595004

Carlos Stiven Mora Hoyos 20261595003

Sergio Santiago Duarte Rojas 20261595002

Curso: Analítica de datos

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia
25 de mayo de 2026

Índice

1. Actividad 1: Backpropagation con función tangente hiperbólica	3
1.1. ¿Por qué usar la tangente hiperbólica?	3
1.2. El algoritmo paso a paso	3
1.3. Ejemplo numérico: XOR con red 2-2-1	3
2. Actividad 2: Diferencias entre Naïve Bayes y red TAN	4
2.1. Naïve Bayes: cada atributo por su cuenta	4
2.2. TAN: permitir una conexión entre atributos	5
2.3. Comparación directa	5
2.4. Ejemplo NB: clasificación de correos	5
2.5. Ejemplo TAN: diagnóstico médico	6
3. Actividad 3: Tolerancia al ruido en redes neuronales	6
3.1. ¿Por qué toleran el ruido?	6
3.2. Ejemplo: reconocimiento de un dígito con ruido	6
4. Actividad 4: Heurísticas para la tasa de aprendizaje	7
4.1. Heurística 1: Decaimiento de la tasa de aprendizaje	7
4.2. Heurística 2: Tasa de aprendizaje adaptativa (escalamiento por coordenadas)	7
4.3. Heurística 3: Programación cíclica de la tasa de aprendizaje (CLR)	8
5. Actividad 5: Diagrama de dispersión y clustering	8
5.1. Principio de funcionamiento	8
5.2. Ejemplo ilustrativo	9
5.3. Uso del diagrama para seleccionar algoritmos	9
6. Actividad 6: Clustering para detección de anomalías	10
6.1. Fundamento teórico	10
6.2. Enfoque práctico: Clustering + umbral de distancia	10
6.3. Ventajas y limitaciones	10
7. Actividad 7: Métodos jerárquicos en análisis de grupos	11
7.1. Variantes y criterios de enlace	11
7.2. Ejemplo: cinco puntos en $\mathbb{R}^2$	11
8. Actividad 8: SOM (Kohonen) vs ART2	12
8.1. SOM: preservación topológica sobre rejilla fija	12
8.2. ART2: plasticidad-estabilidad con vigilancia adaptativa	12
8.3. Comparación	13
9. Actividad 9: Agrupación incremental	14
9.1. Algoritmo BIRCH	14
9.2. Ejemplo: monitoreo de transacciones bancarias	14
10. Actividad 10: Distancias de similitud para agrupación	15
10.1. Distancia euclidiana canónica	15
10.2. Distancia de Manhattan	16
10.3. Distancia del supremo	16
10.4. Distancia de Canberra	16
10.5. Distancia binaria	16
10.6. Distancia de Mahalanobis	17
10.7. Distancia de Hamming	17

1. Actividad 1: Backpropagation con función tangente hiperbólica

Una red neuronal aprende en dos pasos: primero calcula una salida con los datos de entrada (*forward*), y luego compara esa salida con la respuesta correcta para ajustar sus pesos (*backward*). Este proceso se llama retropropagación o *backpropagation* [11], y la forma en que se ajustan los pesos depende directamente de la función de activación elegida.

1.1. ¿Por qué usar la tangente hiperbólica?

La tangente hiperbólica transforma cualquier valor de entrada a un número entre -1 y $+1$:

$$\varphi(v) = \tanh(v) = \frac{e^v - e^{-v}}{e^v + e^{-v}}. \quad (1)$$

Su principal ventaja frente a la sigmoide (que va de 0 a 1) es que sus salidas están centradas en cero, lo que produce ajustes de pesos más efectivos y una convergencia más rápida [1]. Además, su derivada tiene una forma muy conveniente: si ya se conoce la salida $y = \tanh(v)$, la derivada se calcula sin necesidad de reevaluar exponenciales:

$$\varphi'(v) = 1 - y^2. \quad (2)$$

1.2. El algoritmo paso a paso

Sea una red con L capas, donde $w_{ij}^{(\ell)}$ es el peso que conecta la neurona j de la capa anterior con la neurona i de la capa ℓ , $b_i^{(\ell)}$ es el sesgo, y η es la tasa de aprendizaje (qué tan grande es cada ajuste).

Paso 1 — Forward: se calcula la salida de cada neurona, capa por capa:

$$v_i^{(\ell)} = \sum_j w_{ij}^{(\ell)} y_j^{(\ell-1)} + b_i^{(\ell)}, \quad y_i^{(\ell)} = \tanh(v_i^{(\ell)}). \quad (3)$$

Paso 2 — Error: se mide qué tan lejos está la salida de la respuesta correcta:

$$E = \frac{1}{2} \sum_i (d_i - y_i^{(L)})^2.$$

Paso 3 — Backward: se calcula la “culpa” (δ) de cada neurona en el error. En la capa de salida, la culpa es directa; en las capas ocultas, se reparte proporcionalmente a los pesos:

$$\delta_i^{(L)} = (d_i - y_i^{(L)}) (1 - [y_i^{(L)}]^2), \quad (4)$$

$$\delta_i^{(\ell)} = (1 - [y_i^{(\ell)}]^2) \sum_k \delta_k^{(\ell+1)} w_{ki}^{(\ell+1)}, \quad \ell < L. \quad (5)$$

Paso 4 — Actualización: cada peso se ajusta en la dirección que reduce el error:

$$w_{ij}^{(\ell)} \leftarrow w_{ij}^{(\ell)} + \eta \delta_i^{(\ell)} y_j^{(\ell-1)}, \quad b_i^{(\ell)} \leftarrow b_i^{(\ell)} + \eta \delta_i^{(\ell)}. \quad (6)$$

1.3. Ejemplo numérico: XOR con red 2-2-1

Para ver el algoritmo en acción, se entrena una red pequeña (2 entradas, 2 neuronas ocultas, 1 salida) para aprender la función XOR. Como $\tanh$ produce valores entre -1 y $+1$, las etiquetas se codifican en ese rango.

Se presentan los pesos iniciales (valores pequeños aleatorios) y se muestra una iteración completa con el patrón $(x_1, x_2) = (-1, +1)$, cuya salida deseada es $d = +1$, y $\eta = 0,5$.

Tabla 1: Pesos iniciales de la red.

Neurona	w_{i1}	w_{i2}	b_i
h_1 (oculta)	0,30	0,50	0,10
h_2 (oculta)	-0,20	0,40	-0,10
o_1 (salida)	0,40	-0,30	0,20

Forward — se calculan las salidas de cada neurona:

$$\begin{aligned}
 v_1^{(1)} &= 0,30(-1) + 0,50(+1) + 0,10 = 0,30, & y_1^{(1)} &= \tanh(0,30) = 0,2913, \\
 v_2^{(1)} &= -0,20(-1) + 0,40(+1) - 0,10 = 0,50, & y_2^{(1)} &= \tanh(0,50) = 0,4621, \\
 v_1^{(2)} &= 0,40(0,2913) - 0,30(0,4621) + 0,20 = 0,1779, & y_1^{(2)} &= \tanh(0,1779) = 0,1762.
 \end{aligned}$$

La red responde 0,1762 cuando debería responder +1. Error: $E = \frac{1}{2}(1 - 0,1762)^2 = 0,3393$.

Backward — se calcula la culpa de cada neurona:

$$\begin{aligned}
 \delta_1^{(2)} &= (1 - 0,1762)(1 - 0,1762^2) = 0,7983, \\
 \delta_1^{(1)} &= (1 - 0,2913^2) \times 0,7983 \times 0,40 = 0,2922, \\
 \delta_2^{(1)} &= (1 - 0,4621^2) \times 0,7983 \times (-0,30) = -0,1884.
 \end{aligned}$$

Actualización — cada peso se mueve para reducir el error:

Tabla 2: Pesos antes y después de una iteración.

Peso	Antes	Después	Cambio
$w_{11}^{(1)}$	0,3000	0,1539	-0,1461
$w_{12}^{(1)}$	0,5000	0,6461	+0,1461
$b_1^{(1)}$	0,1000	0,2461	+0,1461
$w_{21}^{(1)}$	-0,2000	-0,1058	+0,0942
$w_{22}^{(1)}$	0,4000	0,3058	-0,0942
$b_2^{(1)}$	-0,1000	-0,1942	-0,0942
$w_{11}^{(2)}$	0,4000	0,5163	+0,1163
$w_{12}^{(2)}$	-0,3000	-0,1156	+0,1844
$b_1^{(2)}$	0,2000	0,5992	+0,3992

Los pesos de la capa de salida cambian más que los de la capa oculta, porque están más cerca del error. A medida que el error se propaga hacia atrás, se atenúa al multiplicarse por $(1 - y^2)$ en cada capa. Repitiendo este proceso con los cuatro patrones XOR durante múltiples épocas, la red converge en 500–2000 iteraciones [1].

2. Actividad 2: Diferencias entre Naïve Bayes y red TAN

Tanto Naïve Bayes (NB) como TAN son clasificadores que usan el teorema de Bayes para predecir a qué clase pertenece una observación, a partir de sus atributos. La diferencia clave está en cómo tratan las relaciones entre atributos [3].

2.1. Naïve Bayes: cada atributo por su cuenta

NB asume que los atributos son independientes entre sí una vez que se conoce la clase. Es decir, saber el valor de un atributo no da información sobre otro. La clasificación se hace eligiendo la clase más

probable:

$$\hat{c} = \arg \max_c P(C=c) \prod_{i=1}^n P(X_i | C=c). \quad (7)$$

En la Figura 1 se ve su estructura: la clase C influye en todos los atributos, pero entre atributos no hay conexión alguna.

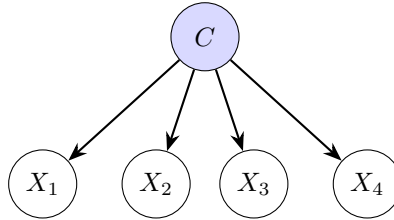


Figura 1: Estructura de Naïve Bayes: solo la clase influye en los atributos.

2.2. TAN: permitir una conexión entre atributos

TAN (*Tree Augmented Naïve Bayes*) reconoce que en la realidad los atributos SÍ pueden estar relacionados. Por eso permite que cada atributo dependa, además de la clase, de *otro atributo* [3]:

$$P(\mathbf{X} | C) = \prod_{i=1}^n P(X_i | C, \text{Pa}(X_i)). \quad (8)$$

Para decidir qué atributos conectar, se usa el algoritmo de Chow–Liu [4]: se mide cuánta información comparte cada par de atributos (información mutua condicional) y se construye un árbol con las conexiones más fuertes.

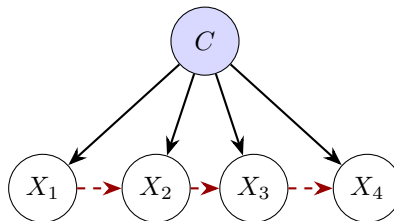


Figura 2: Estructura TAN: las aristas rojas muestran dependencias entre atributos.

2.3. Comparación directa

Tabla 3: Diferencias entre NB y TAN.

Criterio	Naïve Bayes	TAN
Relación entre atributos	Ignorada por completo	Captura la más importante
Cantidad de parámetros	Pocos (simple)	Más (por las conexiones extra)
Riesgo de sobreajuste	Bajo	Moderado
Cuándo conviene	Pocos datos, atributos poco relacionados	Datos suficientes, atributos correlacionados

2.4. Ejemplo NB: clasificación de correos

Se quiere clasificar correos como SPAM o NO-SPAM usando tres palabras clave: X_1 = “oferta”, X_2 = “urgente”, X_3 = “adjunto”. De 8 correos de entrenamiento (4 spam, 4 no-spam), se estiman las probabilidades:

	$P(X_i=1 \mid \text{spam})$	$P(X_i=1 \mid \text{no-spam})$
Oferta	3/4	1/4
Urgente	3/4	1/4
Adjunto	1/4	3/4

Para un correo nuevo con “oferta” y “urgente” pero sin “adjunto” ($X_1=1, X_2=1, X_3=0$):

$$P(\text{spam} \mid \mathbf{X}) \propto 0,5 \times \frac{3}{4} \times \frac{3}{4} \times \frac{3}{4} = 0,2109,$$

$$P(\text{no-spam} \mid \mathbf{X}) \propto 0,5 \times \frac{1}{4} \times \frac{1}{4} \times \frac{1}{4} = 0,0078.$$

Normalizando: $P(\text{spam}) = 96,4\%$. Se clasifica como **spam**. NB funciona bien aquí porque simplemente multiplica las probabilidades de cada palabra por separado.

2.5. Ejemplo TAN: diagnóstico médico

Un médico quiere diagnosticar una enfermedad usando tres síntomas: X_1 = fiebre, X_2 = tos, X_3 = dolor de cabeza. En la práctica, fiebre y tos suelen aparecer juntas (una infección causa ambas), pero NB ignora esta relación.

TAN mide cuánto se relacionan los síntomas entre sí: la conexión fiebre–tos es fuerte ($I = 0,35$), fiebre–dolor es moderada ($I = 0,12$), y tos–dolor es débil ($I = 0,08$). El árbol resultante conecta la fiebre con los otros dos síntomas:

$$P(\mathbf{X} \mid C) = P(X_1 \mid C) \cdot P(X_2 \mid C, X_1) \cdot P(X_3 \mid C, X_1).$$

Esto permite distinguir, por ejemplo, que la probabilidad de tener tos *dado que hay fiebre* es distinta a tener tos *sin fiebre* — una diferencia clínicamente importante que NB no puede capturar [3].

3. Actividad 3: Tolerancia al ruido en redes neuronales

Las redes neuronales artificiales pueden funcionar correctamente incluso cuando los datos de entrada contienen errores o imperfecciones. Esta capacidad no es casualidad: se debe a cómo están construidas [2].

3.1. ¿Por qué toleran el ruido?

1. **El conocimiento está repartido, no concentrado.** La información que aprende una red no se guarda en una sola neurona, sino que se distribuye en miles de conexiones (pesos). Si un dato de entrada llega con errores, las demás conexiones compensan. Es como una pared de ladrillos: si un ladrillo se daña, la pared no se cae.
2. **Las funciones de activación suavizan las perturbaciones.** Funciones como tanh y sigmoide son continuas y suaves: un cambio pequeño en la entrada produce un cambio pequeño en la salida, nunca un salto abrupto. Matemáticamente, su derivada nunca supera 1, lo que significa que el ruido se *reduce* al pasar por cada capa de la red.
3. **La red aprende patrones, no datos individuales.** Durante el entrenamiento, la red extrae las regularidades del conjunto de datos (la “señal”) e ignora las variaciones aleatorias (el “ruido”). Técnicas como *dropout* [5] refuerzan esta capacidad al apagar neuronas al azar durante el entrenamiento, forzando a la red a no depender de ninguna neurona individual.

3.2. Ejemplo: reconocimiento de un dígito con ruido

Se entrena una red (25 entradas, 10 ocultas, 10 salidas) para reconocer dígitos en imágenes de 5×5 píxeles. Se le muestra el dígito “7” con distintos niveles de ruido aleatorio (σ):

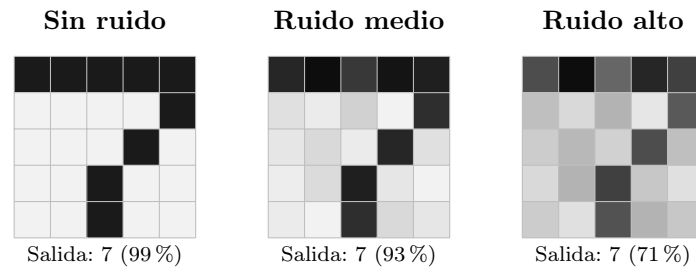


Figura 3: El dígito “7” con niveles crecientes de ruido. La red baja su confianza pero sigue acertando.

Tabla 4: Precisión de la red ante distintos niveles de ruido.

Nivel de ruido (σ)	Precisión	Confianza
0,00 (sin ruido)	100 %	0,99
0,10 (bajo)	98 %	0,95
0,20 (moderado)	90 %	0,84
0,30 (alto)	78 %	0,71
0,50 (extremo)	45 %	0,42

Lo importante es que la precisión baja *gradualmente*, no de golpe. Con un 20 % de ruido, la red todavía acierta el 90 % de las veces. Un sistema basado en reglas exactas (como comparar píxel por píxel con una plantilla) fallaría ante la mínima variación. La red, en cambio, reconoce la “forma general” del dígito y tolera imperfecciones en los detalles [2].

4. Actividad 4: Heurísticas para la tasa de aprendizaje

La tasa de aprendizaje (η) es uno de los hiperparámetros más críticos en el entrenamiento de redes neuronales mediante retropropagación. Un valor excesivamente grande provoca oscilaciones y divergencia; uno demasiado pequeño ralentiza la convergencia o atrapa el optimizador en mínimos locales. A continuación se presentan tres heurísticas ampliamente adoptadas.

4.1. Heurística 1: Decaimiento de la tasa de aprendizaje

Consiste en reducir progresivamente η a medida que avanzan las épocas. La idea central es permitir pasos grandes al inicio (exploración rápida del espacio de pesos) y pasos pequeños al final (refinamiento cercano al óptimo).

Formulación matemática. Dos variantes clásicas son:

1. **Decaimiento exponencial:**

$$\eta(t) = \eta_0 \cdot e^{-\lambda t}, \quad (9)$$

donde η_0 es la tasa inicial, $\lambda > 0$ es la tasa de decaimiento y t es el número de época.

2. **Decaimiento por etapas (step decay):**

$$\eta(t) = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}, \quad (10)$$

donde $\gamma \in (0, 1)$ es el factor de reducción y s es el período (cada cuántas épocas se reduce la tasa).

4.2. Heurística 2: Tasa de aprendizaje adaptativa (escalamiento por coordenadas)

En lugar de usar una tasa global, se adapta una tasa independiente para cada parámetro (peso) según la historia de sus gradientes. Este principio subyace en algoritmos como AdaGrad, RMSprop y Adam.

Formulación de AdaGrad. Sea $g_{t,i}$ el gradiente del parámetro i en la época t . Se acumula el cuadrado del gradiente:

$$G_{t,i} = G_{t-1,i} + g_{t,i}^2, \quad G_{0,i} = 0, \quad (11)$$

y la actualización del peso w_i es:

$$w_{t+1,i} = w_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \varepsilon}} g_{t,i}, \quad \varepsilon \approx 10^{-8}. \quad (12)$$

Parámetros con gradientes históricamente grandes reciben una tasa efectiva menor; parámetros con gradientes pequeños mantienen una tasa mayor, favoreciendo la convergencia en direcciones poco exploradas.

4.3. Heurística 3: Programación cíclica de la tasa de aprendizaje (CLR)

Propuesta por Smith [6], esta heurística varía la tasa de aprendizaje entre un límite inferior $\eta_{\min}$ y uno superior $\eta_{\max}$ de forma periódica, en lugar de monótonamente decreciente. El ciclo puede ser triangular, sinusoidal o exponencial.

Formulación triangular. Para un ciclo de longitud T :

$$\eta(t) = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \cdot \max\left(0, 1 - \left\lfloor \frac{t}{T/2} - 1 \right\rfloor\right). \quad (13)$$

Esta programación ayuda a escapar de mínimos locales y valles de pérdida planos, acelerando la convergencia final.

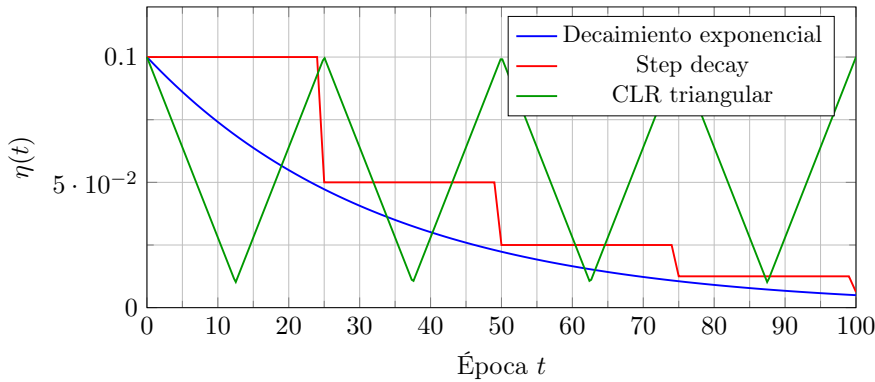


Figura 4: Comportamiento de tres heurísticas para la tasa de aprendizaje durante 100 épocas.

La figura 4 ilustra cómo cada estrategia modifica la tasa a lo largo del entrenamiento. La elección depende del problema, pero en la práctica combinaciones como *Adam con warm-up* o *SGD con momentum + step decay* son estándares robustos.

5. Actividad 5: Diagrama de dispersión y clustering

El análisis de grupos o *clustering* busca descubrir estructuras latentes en los datos sin supervisión previa. El diagrama de dispersión (*scatter plot*) es una herramienta exploratoria esencial porque permite visualizar la distribución espacial de las observaciones y, en muchos casos, identificar visualmente la existencia de conglomerados naturales.

5.1. Principio de funcionamiento

En un espacio de características de baja dimensionalidad (2D o 3D), los puntos que pertenecen al mismo cluster tienden a agruparse en regiones densas del espacio. El diagrama de dispersión revela:

- **Número tentativo de clusters:** regiones densas y separadas sugieren k grupos.
- **Forma de los clusters:** esféricos, elongados, cóncavos o con ruido de fondo.

- **Valores atípicos:** puntos aislados lejos de cualquier conglomerado.
- **Sobrelapamiento:** clusters que comparten fronteras difusas, lo que indica que algoritmos rígidos como *k*-means pueden fallar.

5.2. Ejemplo ilustrativo

Considere un conjunto de datos bidimensional que representa las compras semanales de clientes en un supermercado: eje *X* = *frecuencia de visitas* y eje *Y* = *gasto promedio por visita*. La figura 5 muestra tres patrones diferenciados.

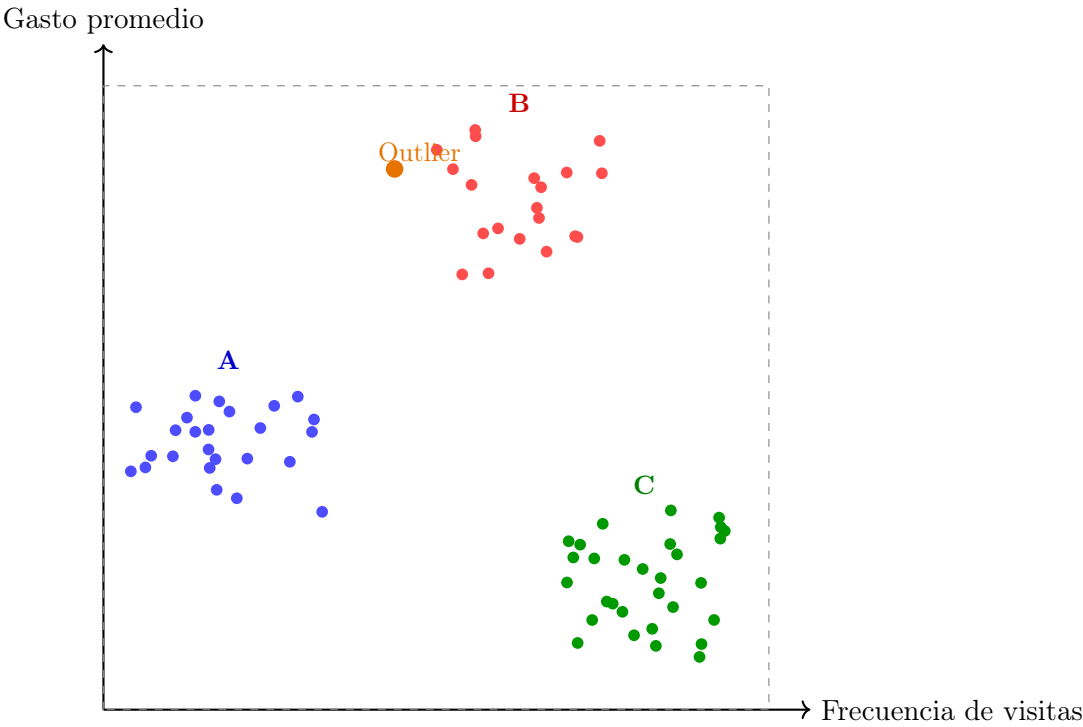


Figura 5: Diagrama de dispersión de clientes según frecuencia de visita y gasto promedio. Se aprecian tres grupos bien diferenciados y un valor atípico (outlier).

5.3. Uso del diagrama para seleccionar algoritmos

Patrón visual	Algoritmo recomendado	Justificación
Esféricos, bien separados	<i>k</i> -means	Minimiza varianza intra-cluster
Formas arbitrarias, densidad	DBSCAN	No asume geometría global
Jerarquía anidada	Clustering jerárquico	Dendrograma muestra niveles
Clusters de densidad variable	HDBSCAN	Robustez a densidades mixtas

Tabla 5: Correspondencia entre patrones visuales en el diagrama de dispersión y algoritmos de clustering.

La tabla 5 resume cómo la morfología observada en el diagrama de dispersión orienta la selección del método de agrupación. En el ejemplo de la figura 5, los tres grupos son aproximadamente esféricos y separados; por tanto, *k*-means con *k* = 3 sería una elección apropiada.

6. Actividad 6: Clustering para detección de anomalías

La detección de valores anómalos (*outliers* o *anomalies*) es una tarea crítica en analítica de datos, con aplicaciones en detección de fraude, mantenimiento predictivo y seguridad de redes. El análisis de grupos proporciona un marco no supervisado para identificar anomalías sin requerir ejemplos etiquetados de comportamiento anómalo.

6.1. Fundamento teórico

La premisa fundamental es que los datos normales forman conglomerados densos y coherentes, mientras que las anomalías son puntos que no encajan en ningún cluster o que residen en regiones de muy baja densidad. Formalmente, sea $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$ y sea $\mathcal{C} = \{C_1, \dots, C_k\}$ una partición obtenida por un algoritmo de clustering. Se definen dos criterios de anomalía [7]:

1. **Distancia al centroide más cercano:** un punto $\mathbf{x}$ es anómalo si

$$\min_{C_j \in \mathcal{C}} \text{dist}(\mathbf{x}, \boldsymbol{\mu}_j) > \theta, \quad (14)$$

donde $\boldsymbol{\mu}_j$ es el centroide del cluster C_j y θ es un umbral predefinido.

2. **Densidad local:** en métodos basados en densidad (DBSCAN, HDBSCAN), los puntos clasificados como *ruido* (label = -1) son directamente anómalos.

6.2. Enfoque práctico: Clustering + umbral de distancia

El procedimiento habitual consta de tres etapas:

1. **Agrupar** los datos con un algoritmo adecuado (por ejemplo, k -means).
2. **Calcular** para cada punto su distancia al centroide de su cluster asignado.
3. **Marcar** como anómalo todo punto cuya distancia exceda un percentil alto (típicamente el percentil 95 o 99) de la distribución de distancias.

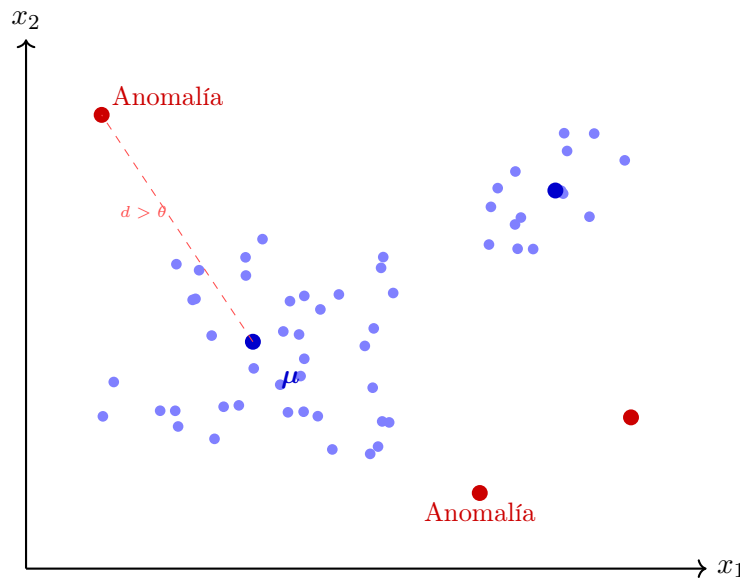


Figura 6: Detección de anomalías mediante clustering y umbral de distancia. Los puntos rojos quedan fuera del radio de tolerancia del cluster más cercano.

6.3. Ventajas y limitaciones

Ventajas:

- No requiere datos etiquetados; es completamente no supervisado.
- Puede adaptarse a distribuciones de datos complejas mediante clustering de densidad.
- Es interpretable: la anomalía se explica por su alejamiento de los centroides o baja densidad local.

Limitaciones:

- Sensibilidad a la elección del número de clusters (k) o del umbral de densidad (ε en DBSCAN).
- Si las anomalías forman un cluster propio, pueden pasar inadvertidas.
- En alta dimensionalidad la maldición de la dimensionalidad distorsiona las distancias y hace necesario reducir dimensionalidad previamente (PCA, t-SNE, UMAP).

En la práctica, el análisis de grupos para detección de anomalías se complementa con técnicas de aprendizaje profundo como *autoencoders*, que aprenden una representación comprimida de los datos normales y detectan anomalías por alto error de reconstrucción [8].

7. Actividad 7: Métodos jerárquicos en análisis de grupos

Los métodos jerárquicos construyen una secuencia anidada de particiones representable mediante un *dendrograma* [14]. A diferencia de k-means, no requieren fijar *a priori* el número de grupos; el analista determina la partición óptima cortando el árbol a la altura que corresponda al salto de distancia más pronunciado.

7.1. Variantes y criterios de enlace

La estrategia **aglomerativa** (*bottom-up*) parte de n grupos singleton y fusiona en cada iteración los dos clusters más próximos hasta que el conjunto queda reducido a uno solo [15]. Su costo es $O(n^2 \log n)$ con estructuras de datos eficientes. La estrategia **divisiva** (*top-down*) subdivide recursivamente el grupo completo; su complejidad $O(2^n)$ la hace impráctica para conjuntos grandes [14].

El criterio de enlace determina la distancia entre clusters y , con ello, el orden de las fusiones:

$$d_{\text{simple}}(C_i, C_j) = \min_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}), \quad (15)$$

$$d_{\text{completo}}(C_i, C_j) = \max_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} d(\mathbf{x}, \mathbf{y}), \quad (16)$$

$$d_{\text{Ward}}(C_i, C_j) = \sqrt{\frac{2|C_i||C_j|}{|C_i| + |C_j|}} \|\bar{\mathbf{x}}_i - \bar{\mathbf{x}}_j\|_2, \quad (17)$$

donde $\bar{\mathbf{x}}_i = \frac{1}{|C_i|} \sum_{\mathbf{p} \in C_i} \mathbf{p}$ es el centroide del cluster C_i . El criterio de Ward [13] minimiza el incremento de varianza intra-cluster en cada fusión y suele producir grupos más compactos y equilibrados que los criterios de enlace simple y completo [15].

7.2. Ejemplo: cinco puntos en $\mathbb{R}^2$

Considérense los puntos $p_1 = (1, 1)$, $p_2 = (2, 1)$, $p_3 = (1.5, 2)$, $p_4 = (8, 8)$ y $p_5 = (9, 7)$. La Tabla 6 muestra la matriz de distancias euclidianas.

Tabla 6: Matriz de distancias euclidianas entre los cinco puntos.

	p_1	p_2	p_3	p_4	p_5
p_1	0	1,00	1,12	9,90	10,00
p_2		0	1,12	9,22	9,22
p_3			0	8,60	9,01
p_4				0	1,41
p_5					0

Bajo enlace simple (15), el algoritmo aglomerativo produce cuatro fusiones en el siguiente orden:

1. $\{p_1\} \cup \{p_2\} \rightarrow C_{12}$, distancia mínima $d = 1,00$.
2. $C_{12} \cup \{p_3\} \rightarrow C_{123}$, distancia $d = 1,12$ (por enlace simple, distancia de p_3 al más cercano de C_{12}).
3. $\{p_4\} \cup \{p_5\} \rightarrow C_{45}$, distancia $d = 1,41$.
4. $C_{123} \cup C_{45}$, distancia $d = 8,60$.

El salto entre $d = 1,41$ y $d = 8,60$ indica la existencia de dos macro-grupos naturales; cualquier corte en el intervalo $(1,41, 8,60)$ recupera exactamente la bipartición $\{p_1, p_2, p_3\}$ y $\{p_4, p_5\}$ [14]. La Figura 7 muestra el dendrograma resultante.

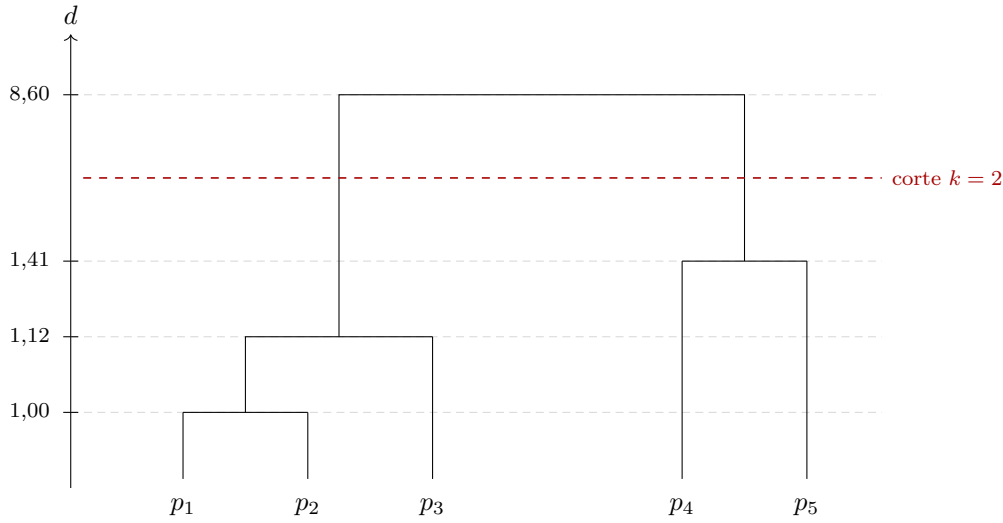


Figura 7: Dendrograma con enlace simple (*escala vertical no proporcional; los valores de d son los reales*). El corte en el intervalo $(1,41; 8,60)$ produce exactamente dos clusters: $\{p_1, p_2, p_3\}$ y $\{p_4, p_5\}$.

En la práctica, la inspección visual del dendrograma permite seleccionar el número de grupos sin asumir ninguna estructura específica, lo que constituye la ventaja definitoria de los métodos jerárquicos frente a algoritmos partitivos [15].

8. Actividad 8: SOM (Kohonen) vs ART2

Los mapas auto-organizativos de Kohonen (SOM) y la red ART2 (*Adaptive Resonance Theory 2*) son redes neuronales no supervisadas para datos continuos, pero responden a objetivos y supuestos fundamentalmente distintos [14].

8.1. SOM: preservación topológica sobre rejilla fija

Un SOM consta de una capa de entrada de dimensión n y una capa de salida organizada en una rejilla de dimensión fija (habitualmente 2D) [16]. Cada neurona j posee un vector de pesos $\mathbf{w}_j \in \mathbb{R}^n$. Ante un patrón $\mathbf{x}$, se identifica la *neurona ganadora* (BMU, *Best Matching Unit*):

$$j^* = \arg \min_j \|\mathbf{x} - \mathbf{w}_j\|,$$

y se actualizan los pesos de j^* y sus vecinas según:

$$\mathbf{w}_j(t+1) = \mathbf{w}_j(t) + \eta(t) h_{j,j^*}(t) [\mathbf{x}(t) - \mathbf{w}_j(t)],$$

donde $\eta(t)$ es la tasa de aprendizaje y $h_{j,j^*}(t) = \exp(-\|r_j - r_{j^*}\|^2 / 2\sigma^2(t))$ es la función de vecindad gaussiana [16]. El resultado es un mapa que preserva la topología del espacio de entrada: patrones similares activan neuronas adyacentes en la rejilla.

8.2. ART2: plasticidad-estabilidad con vigilancia adaptativa

ART2 aborda explícitamente el *dilema plasticidad-estabilidad*: la red debe incorporar categorías nuevas sin degradar las ya aprendidas [18]. Su mecanismo central es un parámetro de vigilancia $\rho \in (0, 1)$ que controla la granularidad de las categorías [17]. Cuando llega un patrón $\mathbf{x}$, se selecciona la categoría

candidata j^* por mayor activación; si la similitud normalizada no supera el umbral:

$$\frac{\|\mathbf{x} \wedge \mathbf{w}_{j^*}\|}{\|\mathbf{x}\|} < \rho,$$

la categoría es rechazada y se activa la siguiente candidata [17]. Si ninguna supera el umbral, se crea automáticamente una categoría nueva, dejando las existentes intactas, lo que garantiza la estabilidad del conocimiento previo [18].

8.3. Comparación

Tabla 7: Diferencias fundamentales entre SOM [16] y ART2 [17].

Criterio	SOM	ART2
Número de neuronas	Fijo, definido antes del entrenamiento	Dinámico; crece conforme llegan nuevos patrones
Preservación topológica	Sí; la proximidad en la rejilla refleja similitud en el espacio de entrada	No
Dilema plasticidad-estabilidad	No abordado; el reentrenamiento puede borrar representaciones previas	Resuelto explícitamente mediante ρ
Parámetro clave	Radio de vecindad $\sigma(t)$ y tasa $\eta(t)$	Parámetro de vigilancia ρ
Tipo de aprendizaje	Competitivo cooperativo; actualiza neurona ganadora y vecinas	Competitivo puro; solo se actualiza la categoría seleccionada
Aprendizaje online	Posible, pero con degradación si la distribución cambia	Estable frente a nuevos conceptos; no requiere reentrenamiento global
Aplicación típica	Visualización, reducción de dimensionalidad, análisis exploratorio	Clasificación en tiempo real, detección de conceptos emergentes
Sensibilidad al orden de presentación	Baja; el mapa converge hacia una representación estable con entrenamiento suficiente, independientemente del orden	Alta; el orden en que llegan los patrones determina qué categorías se crean primero y puede producir arquitecturas distintas ante los mismos datos reordenados
Maldición de la dimensionalidad	Pronunciada; la rejilla fija de neuronas no escala con espacios de alta dimensión, y la distancia euclidiana pierde discriminación [22]	Moderada; el parámetro ρ opera sobre normas normalizadas, aunque $\ \mathbf{x}\ $ también se degrada en dimensiones elevadas

En síntesis, SOM es preferible cuando se busca una representación visual compacta de la estructura de los datos [16], mientras que ART2 resulta más adecuado en escenarios donde el número de categorías es desconocido y el sistema debe reconocer patrones nuevos sin comprometer el conocimiento previo [17, 18].

9. Actividad 9: Agrupación incremental

La *agrupación incremental* (también denominada *online* o en flujo de datos) construye y actualiza el modelo de clusters a medida que llegan nuevas observaciones, sin acceso al historial completo [20]. Resulta indispensable cuando el volumen de datos supera la capacidad de memoria disponible, cuando la distribución subyacente evoluciona con el tiempo (*concept drift*), o cuando se requiere respuesta en tiempo real [20].

9.1. Algoritmo BIRCH

BIRCH (*Balanced Iterative Reducing and Clustering using Hierarchies*) resume cada subconjunto de puntos mediante una *Característica de Clustering* (CF) [19]:

$$CF = (N, \mathbf{LS}, SS),$$

donde N es el número de puntos del subconjunto, $\mathbf{LS} = \sum_{i=1}^N \mathbf{x}_i$ es la suma lineal y $SS = \sum_{i=1}^N \|\mathbf{x}_i\|^2$ la suma de cuadrados. Las CF satisfacen la propiedad de aditividad:

$$CF_1 + CF_2 = (N_1 + N_2, \mathbf{LS}_1 + \mathbf{LS}_2, SS_1 + SS_2),$$

lo que permite incorporar un nuevo punto $\mathbf{x}$ actualizando únicamente la hoja del árbol CF-Tree más cercana en $O(1)$ [19]. Si la distancia entre $\mathbf{x}$ y el centroide de dicha hoja excede el umbral T , se genera un nuevo nodo; de lo contrario, la CF del nodo se actualiza incrementalmente.

Observación 9.1 (Limitación geométrica de BIRCH). Al resumir cada subconjunto mediante su centroide, radio y diámetro (derivados de $\mathbf{LS}$ y SS), BIRCH asume implícitamente que los clusters tienen forma esférica, de manera análoga a k-means [19]. Esta suposición produce resúmenes inadecuados ante geometrías irregulares, cóncavas o con densidad variable: dos semicírculos concéntricos, por ejemplo, quedarían colapsados en la misma CF. En tales escenarios conviene recurrir a algoritmos basados en densidad como DenStream o CluStream, que no imponen restricciones geométricas al definir los micro-clusters [20].

9.2. Ejemplo: monitoreo de transacciones bancarias

Considérese un flujo de transacciones caracterizadas por monto (\$) y hora del día. El modelo aprende inicialmente dos perfiles: transacciones diurnas de bajo monto (C_A , horas 8–17) y nocturnas de alto monto (C_B , horas 19–24), como muestra la Figura 8.

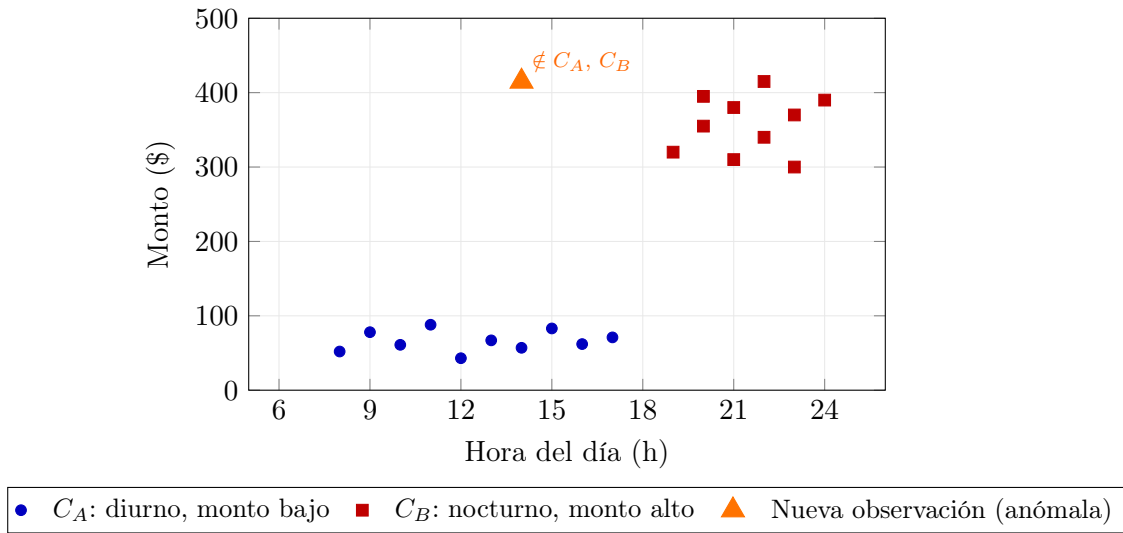


Figura 8: Flujo de transacciones bancarias procesadas por BIRCH. Los clusters C_A (diurno, monto bajo) y C_B (nocturno, monto alto) representan el conocimiento incremental acumulado. La nueva observación (14h, \$415) no supera el umbral de similitud T con ninguno de los dos perfiles existentes, por lo que BIRCH crea un tercer nodo CF [19].

Cuando llega una transacción a las 14h con monto de \$415, su distancia al centroide de C_A (monto $\approx$ \$66, horas 8–17) supera el umbral T por exceso de monto, y su distancia al centroide de C_B (horas 19–24) lo supera por discrepancia horaria. BIRCH crea una nueva hoja en el árbol CF, dando origen a un tercer perfil potencialmente fraudulento, sin necesidad de reanálisis global [19]. La capacidad de adaptarse a patrones emergentes sin reentrenar desde cero es la propiedad que distingue a los algoritmos incrementales en entornos de datos en flujo [20].

10. Actividad 10: Distancias de similitud para agrupación

Las métricas de distancia cuantifican la disimilitud entre observaciones y determinan, en gran medida, la calidad y morfología de los clusters obtenidos [14]. La selección de una distancia adecuada debe considerar la naturaleza de las variables (continuas, binarias, nominales), la escala de las mismas y la presencia de correlaciones entre atributos [22].

A lo largo de esta sección se emplea el par de vectores $\mathbf{x} = (4, 3)^\top$, $\mathbf{y} = (0, 0)^\top$ salvo indicación contraria, lo que permite comparar directamente los valores numéricos entre métricas sobre los mismos datos.

10.1. Distancia euclidiana canónica

Definición 10.1 ([21]). Sean $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$. La *distancia euclidiana canónica* se define como:

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} = \|\mathbf{x} - \mathbf{y}\|_2.$$

Corresponde a la norma L^2 del vector diferencia y representa la distancia rectilínea entre dos puntos en el espacio euclídeo. En agrupación, minimizar la suma de distancias euclidianas al cuadrado hacia el centroide es el criterio que subyace en k-means [21]. Su principal limitación es la sensibilidad a la escala: variables con mayor varianza dominan el cómputo, por lo que se recomienda estandarizar los datos [22].

Ejemplo. Con $\mathbf{x} = (4, 3)^\top$ e $\mathbf{y} = (0, 0)^\top$:

$$d_E = \sqrt{(4 - 0)^2 + (3 - 0)^2} = \sqrt{16 + 9} = 5.$$

10.2. Distancia de Manhattan

Definición 10.2 ([21]). La *distancia de Manhattan* (norma L^1 , distancia taxicab) es:

$$d_M(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i| = \|\mathbf{x} - \mathbf{y}\|_1.$$

Suma las diferencias absolutas coordenada a coordenada; al no elevar al cuadrado las diferencias, es más robusta a valores atípicos que la distancia euclidiana [21]. Se emplea en variantes robustas de k-means (k-medoides) y en contextos donde la distribución de los datos presenta colas pesadas.

Ejemplo. Con $\mathbf{x} = (4, 3)^\top$ e $\mathbf{y} = (0, 0)^\top$:

$$d_M = |4 - 0| + |3 - 0| = 4 + 3 = 7.$$

10.3. Distancia del supremo

Definición 10.3 ([21]). La *distancia del supremo* (distancia de Chebyshev, norma L^∞) es:

$$d_\infty(\mathbf{x}, \mathbf{y}) = \max_{1 \leq i \leq n} |x_i - y_i| = \|\mathbf{x} - \mathbf{y}\|_\infty = \lim_{p \rightarrow \infty} \|\mathbf{x} - \mathbf{y}\|_p.$$

Captura la mayor discrepancia en una sola dimensión, ignorando las demás. Su uso en clustering es apropiado cuando el peor caso en una variable resulta más relevante que la discrepancia acumulada [22]; por ejemplo, en control de calidad donde se exige tolerancia máxima por dimensión independiente.

Ejemplo. Con $\mathbf{x} = (4, 3)^\top$ e $\mathbf{y} = (0, 0)^\top$:

$$d_\infty = \max(|4 - 0|, |3 - 0|) = \max(4, 3) = 4.$$

10.4. Distancia de Canberra

Definición 10.4 ([23]). La *distancia de Canberra* es:

$$d_C(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n \frac{|x_i - y_i|}{|x_i| + |y_i|},$$

con la convención de que el i -ésimo término es 0 cuando $x_i = y_i = 0$.

Normaliza cada término por la magnitud total de las coordenadas involucradas, por lo que es invariante a escala y más sensible a diferencias en valores pequeños que la distancia euclidiana [27]. Su rango es $[0, n]$. En clustering, resulta particularmente adecuada para datos de conteo, perfiles de abundancia y detección de intrusiones [27].

Ejemplo. Con $\mathbf{x} = (4, 3)^\top$ e $\mathbf{y} = (2, 1)^\top$:

$$d_C = \frac{|4 - 2|}{|4| + |2|} + \frac{|3 - 1|}{|3| + |1|} = \frac{2}{6} + \frac{2}{4} = 0,333 + 0,500 = 0,833.$$

10.5. Distancia binaria

Definición 10.5 ([24]). Para vectores binarios $\mathbf{x}, \mathbf{y} \in \{0, 1\}^n$, se definen los conteos:

$$n_{11} = |\{i : x_i = 1, y_i = 1\}|, \quad n_{10} = |\{i : x_i = 1, y_i = 0\}|, \quad n_{01} = |\{i : x_i = 0, y_i = 1\}|.$$

La *distancia binaria* (equivalente a la distancia de Jaccard) es:

$$d_B(\mathbf{x}, \mathbf{y}) = \frac{n_{10} + n_{01}}{n_{11} + n_{10} + n_{01}}.$$

Mide la proporción de discordancias respecto del total de posiciones en que al menos uno de los vectores presenta el atributo [24]. Los ceros conjuntos (n_{00}) se excluyen por no aportar información

de co-presencia. Es la distancia de referencia para atributos de presencia/ausencia en clustering de documentos, perfiles genéticos y datos de encuestas dicotómicas.

Ejemplo. Con $\mathbf{x} = (1, 0, 1, 1, 0)$ e $\mathbf{y} = (1, 1, 0, 1, 0)$:

Posición	1	2	3	4	5
x_i	1	0	1	1	0
y_i	1	1	0	1	0

Se tiene $n_{11} = 2$, $n_{10} = 1$, $n_{01} = 1$, $n_{00} = 1$, luego:

$$d_B = \frac{1+1}{2+1+1} = \frac{2}{4} = 0,5.$$

10.6. Distancia de Mahalanobis

Definición 10.6 ([25]). Sea $\mathbf{S} \in \mathbb{R}^{n \times n}$ la matriz de covarianza del conjunto de datos, con $\mathbf{S}$ invertible. La *distancia de Mahalanobis* entre $\mathbf{x}$ e $\mathbf{y}$ es:

$$d_{\text{Mah}}(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top \mathbf{S}^{-1} (\mathbf{x} - \mathbf{y})}.$$

Generaliza la distancia euclidiana teniendo en cuenta la escala y las correlaciones entre variables: pares de coordenadas fuertemente correlacionadas reciben menor distancia efectiva que si se ignorara su covarianza [25]. Cuando $\mathbf{S} = \mathbf{I}$, se reduce a la distancia euclidiana. Es la distancia implícita en los modelos de mezcla gaussiana (GMM), donde cada componente puede tener covarianza propia [21].

Ejemplo. Sean $\mathbf{x} = (4, 3)^\top$, $\mathbf{y} = (0, 0)^\top$ y

$$\mathbf{S} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}, \quad \mathbf{S}^{-1} = \frac{1}{8} \begin{pmatrix} 3 & -2 \\ -2 & 4 \end{pmatrix}.$$

Con $\boldsymbol{\delta} = \mathbf{x} - \mathbf{y} = (4, 3)^\top$:

$$d_{\text{Mah}}^2 = \frac{1}{8} (4, 3) \begin{pmatrix} 3 & -2 \\ -2 & 4 \end{pmatrix} \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \frac{1}{8} (4, 3) \begin{pmatrix} 6 \\ 4 \end{pmatrix} = \frac{24+12}{8} = \frac{36}{8} = 4,5,$$

de donde $d_{\text{Mah}} = \sqrt{4,5} \approx 2,12$. La distancia euclidiana del mismo par era 5; la reducción refleja que las coordenadas están positivamente correlacionadas.

10.7. Distancia de Hamming

Definición 10.7 ([26]). Para dos vectores (o cadenas) de longitud n sobre un alfabeto discreto, la *distancia de Hamming* cuenta el número de posiciones en que difieren:

$$d_H(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n \mathbf{1}[x_i \neq y_i].$$

Formulada originalmente para la detección y corrección de errores en códigos binarios [26], se aplica a cualquier alfabeto discreto. Su rango es $[0, n]$; la versión normalizada $d_H/n \in [0, 1]$ permite comparar vectores de longitudes distintas. En clustering, es la distancia natural para variables categóricas codificadas, secuencias de ADN y cadenas de caracteres [14]. A diferencia de la distancia binaria, los ceros conjuntos sí contribuyen al cómputo.

Ejemplo. Con $\mathbf{x} = (1, 0, 1, 1, 0, 1)$ e $\mathbf{y} = (1, 1, 0, 1, 1, 0)$:

Posición	1	2	3	4	5	6
x_i	1	0	1	1	0	1
y_i	1	1	0	1	1	0
Discordancia	—	✓	✓	—	✓	✓

$d_H = 4.$

Referencias

- [1] LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (2012). Efficient BackProp. In *Neural Networks: Tricks of the Trade* (2nd ed.), Lecture Notes in Computer Science, vol. 7700, pp. 9–48. https://doi.org/10.1007/978-3-642-35289-8_3
- [2] Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Prentice Hall. ISBN: 978-0-13-147139-9.
- [3] Friedman, N., Geiger, D., & Goldszmidt, M. (1997). Bayesian network classifiers. *Machine Learning*, 29(2–3), 131–163. <https://doi.org/10.1023/A:1007465528199>
- [4] Chow, C., & Liu, C. (1968). Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3), 462–467. <https://doi.org/10.1109/TIT.1968.1054142>
- [5] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- [6] Smith, L. N. (2017). Cyclical learning rates for training neural networks. *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 464–472. <https://doi.org/10.1109/WACV.2017.58>
- [7] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58. <https://doi.org/10.1145/1541880.1541882>
- [8] Chalapathy, R., & Chawla, S. (2019). Deep learning for anomaly detection: A survey. *arXiv preprint arXiv:1901.03407*. <https://arxiv.org/abs/1901.03407>
- [9] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD'96)*, 226–231.
- [10] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- [11] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>
- [12] Xu, R., & Wunsch, D. (2005). Survey of clustering algorithms. *IEEE Transactions on Neural Networks*, 16(3), 645–678. <https://doi.org/10.1109/TNN.2005.845141>
- [13] Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301), 236–244. <https://doi.org/10.1080/01621459.1963.10500845>
- [14] Jain, A. K., Murty, M. N., & Flynn, P. J. (1999). Data clustering: A review. *ACM Computing Surveys*, 31(3), 264–323. <https://doi.org/10.1145/331499.331504>
- [15] Murtagh, F., & Contreras, P. (2012). Algorithms for hierarchical clustering: an overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 2(1), 86–97. <https://doi.org/10.1002/widm.53>
- [16] Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE*, 78(9), 1464–1480. <https://doi.org/10.1109/5.58325>
- [17] Carpenter, G. A., & Grossberg, S. (1987). ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, 26(23), 4919–4930. <https://doi.org/10.1364/AO.26.004919>
- [18] Grossberg, S. (1987). Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1), 23–63. <https://doi.org/10.1111/j.1551-6708.1987.tb00862.x>
- [19] Zhang, T., Ramakrishnan, R., & Livny, M. (1996). BIRCH: An efficient data clustering method for very large databases. *ACM SIGMOD Record*, 25(2), 103–114. <https://doi.org/10.1145/235968.233324>
- [20] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 44:1–44:37. <https://doi.org/10.1145/2523813>
- [21] Duda, R. O., Hart, P. E., & Stork, D. G. (2001). *Pattern Classification* (2nd ed.). Wiley-

- Interscience. ISBN: 978-0-471-05669-0.
- [22] Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. In *Database Theory — ICDT 2001*, Lecture Notes in Computer Science, vol. 1973, pp. 420–434. https://doi.org/10.1007/3-540-44503-X_27
 - [23] Lance, G. N., & Williams, W. T. (1967). Mixed-data classificatory programs I: Agglomerative systems. *Australian Computer Journal*, 1(1), 15–20.
 - [24] Real, R., & Vargas, J. M. (1996). The probabilistic basis of Jaccard’s index of similarity. *Systematic Biology*, 45(3), 380–385. <https://doi.org/10.1093/sysbio/45.3.380>
 - [25] De Maesschalck, R., Jouan-Rimbaud, D., & Massart, D. L. (2000). The Mahalanobis distance. *Chemometrics and Intelligent Laboratory Systems*, 50(1), 1–18. [https://doi.org/10.1016/S0169-7439\(99\)00047-7](https://doi.org/10.1016/S0169-7439(99)00047-7)
 - [26] Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2), 147–160. <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>
 - [27] Emran, S. M., & Ye, N. (2002). Robustness of chi-square and Canberra distance metrics for computer intrusion detection. *Quality and Reliability Engineering International*, 18(1), 19–28. <https://doi.org/10.1002/qre.441>